

SEMANA 1 · TUTORÍA SINCRÓNICA

Introducción al Aprendizaje Automático

Sábado 19/09/2026 · 3 h MIAR0525 · Maestría en Inteligencia Artificial

Agenda de hoy

Tiempo	Bloque	Animaciones
0:00–0:15	Bienvenida, el hub del curso, evaluación y normas	—
0:15–0:50	¿Qué es aprender de datos? Tipos de aprendizaje y formulación del problema	A1.1
0:50–1:30	El proceso de ML: particiones, validación cruzada y baselines	A1.2 · A1.3
1:30–1:40	Pausa	—
1:40–2:15	Métricas de regresión y clasificación; el costo del error	A1.5 · A1.6
2:15–2:45	Fuga de datos y la solución con Pipeline	A1.4
2:45– 3:00	Tarea 1, dudas y ticket de salida	—

Cómo funciona el curso

- 1 **Prepárate** con videos, manual y prelaboratorio (el lunes).
- 2 **Tutoría** del sábado: dudas, animaciones, casos.
- 3 **Practica** en notebooks, con su versión resuelta y un desafío opcional con IA.
- 4 **Entrega** una tarea por semana (semanas 1–3).

Evaluación

Tareas 1, 2 y 3: **20 % cada una** (60 %)

Examen teórico: **15 %** · Examen práctico integrador: **25 %**

Mínimo aprobatorio: 70/100. Puedes usar IA (agéntica o de chat) en ejercicios y tareas, siempre declarada.

BLOQUE 1 · 0:15–0:50

¿Qué es aprender de datos?

Definición, tipos de aprendizaje y cómo se formula un problema.

Un programa aprende si mejora con la experiencia

Un programa **aprende** de la experiencia **E** respecto de una tarea **T** y una medida de desempeño **P** si su desempeño en T, medido por P, mejora con E. (Mitchell, 1997)

T · tarea

Clasificar cada correo entrante como phishing o legítimo.

E · experiencia

20 000 correos ya etiquetados por el equipo de seguridad.

P · desempeño

Recall de phishing con una tasa de falsas alarmas tolerable.

Reglas escritas frente a reglas aprendidas

Programación tradicional

Datos + **reglas escritas** → respuestas

“Si el remitente no está en la lista blanca y hay un enlace acortado, marcar.”

Aprendizaje automático

Datos + **respuestas** → reglas aprendidas

El modelo descubre las combinaciones que separan phishing de legítimo.

- Usa ML cuando las reglas son demasiadas, cambian o nadie sabe escribirlas.
- **No** uses ML si una regla simple resuelve el problema, si no hay datos representativos o si el error no es tolerable ni explicable.

Tres formas de aprender



Antes de elegir un modelo, formula el problema

- 1 **Unidad de análisis:** ¿qué es una fila?
(un cliente, un correo, una sesión).
- 2 **Variable objetivo:** qué predecir y cómo se etiqueta.
- 3 **Momento de predicción:** cuándo se predice y qué datos existen en ese instante.
- 4 **Métrica y costo del error:** qué es peor, un falso positivo o un falso negativo.
- 5 **Baseline:** qué resultado obtendría una regla trivial.

Caso de la Tarea 1: abandono de clientes

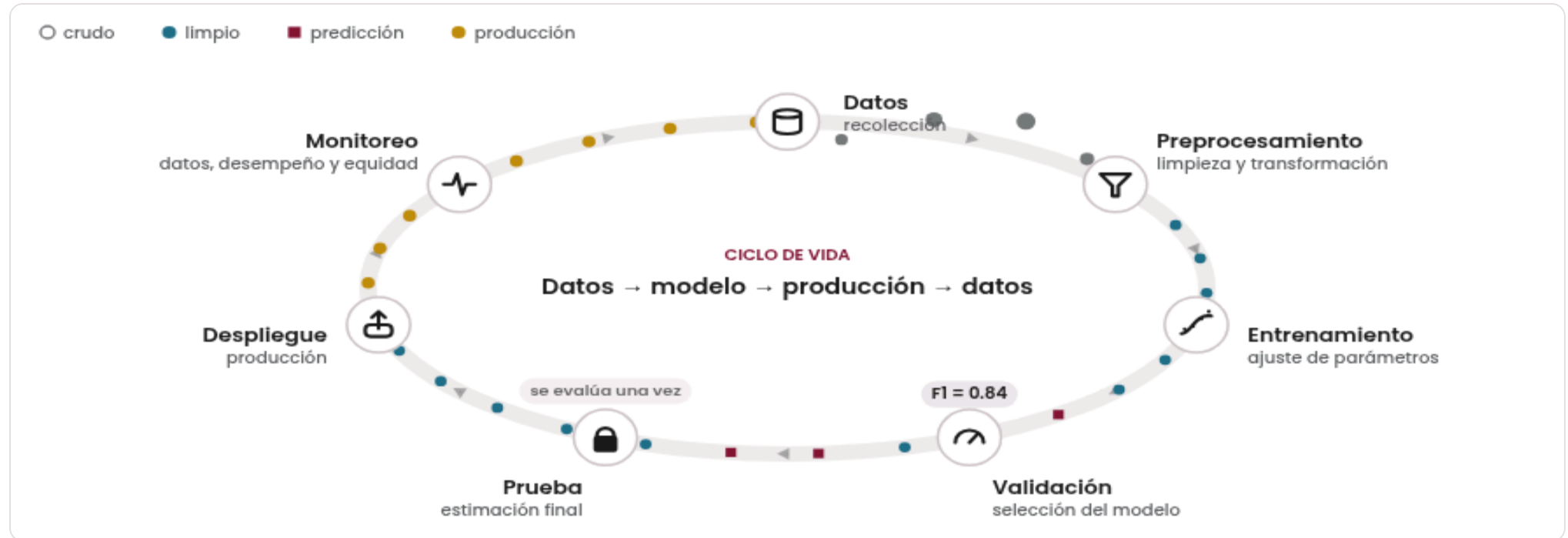
Fila = cliente · objetivo = Churn (sí/no)
· se predice al cierre de cada mes ·
métrica según el costo de retener
frente al de perder un cliente ·
baseline = "nadie abandona".

BLOQUE 2 · 0:50–1:30

El proceso de ML

Del dato crudo al modelo evaluado con honestidad.

Un ciclo, no una línea recta



Entrenamiento, validación y prueba

Dataset en orden de recolección



El piso que tu modelo tiene que superar

Si el 74 % de los clientes no abandona, predecir siempre “no abandona” ya da **74 % de exactitud**.

Un modelo con 76 % de exactitud apenas aprendió algo. Por eso siempre se reporta el baseline junto al modelo.

Python

```
from sklearn.dummy import DummyClassifier

base = DummyClassifier(strategy="most_frequent")
base.fit(X_train, y_train)
base.score(X_val, y_val)    # el piso

# en regresión:
# DummyRegressor(strategy="mean")
```

1:30–1:40

Pausa · 10 minutos

Al volver: ¿cómo sabemos si un modelo es bueno?

BLOQUE 3 · 1:40–2:15

Métricas de evaluación

La métrica correcta depende del costo del error.

¿Cuánto se equivoca en promedio?

MAE

$$\frac{1}{n} \sum |y_i - \hat{y}_i|$$

En las unidades de y.
Robusto a valores
atípicos.

RMSE

$$\sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$$

Penaliza más los errores
grandes.

R²

$$1 - \frac{SS_{res}}{SS_{tot}}$$

Proporción de varianza
explicada respecto de
predecir la media.

Si un error de 1000 USD es más de diez veces peor que uno de 100 USD, usa **RMSE**; si todos los errores pesan igual, **MAE**.

Todo sale de cuatro celdas

	Predice positivo	Predice negativo
Real positivo	VP · acierto	FN · se escapó
Real negativo	FP · falsa alarma	VN · acierto

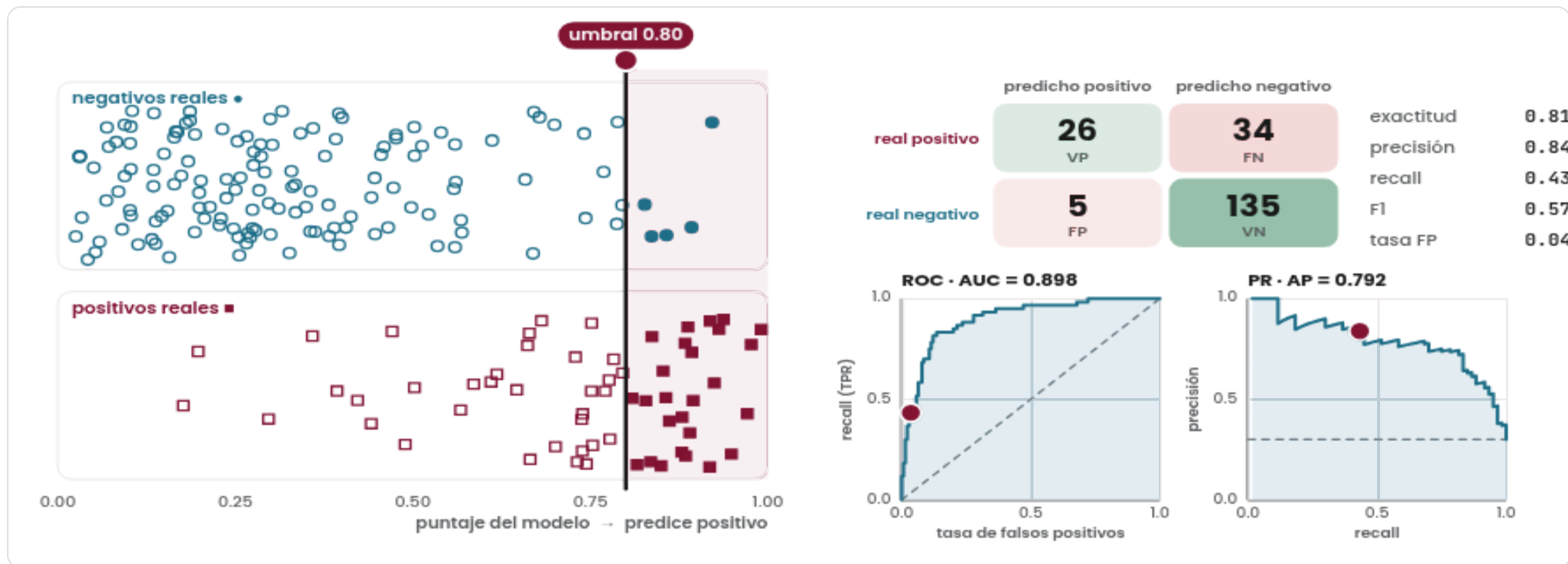
$$\text{precisión} = \frac{VP}{VP+FP}$$

$$\text{recall} = \frac{VP}{VP+FN}$$

$$F_1 = 2 \frac{\text{precisión} \cdot \text{recall}}{\text{precisión} + \text{recall}}$$

La **exactitud** engaña con clases desbalanceadas: con 1 % de fraude, “nunca es fraude” acierta 99 %.

El umbral es una decisión



¿Qué error duele más?

Caso de TI	Falso negativo	Falso positivo	Prioriza
Detección de phishing	Un empleado entrega credenciales	Un correo legítimo va a cuarentena	Recall
Antivirus corporativo	Malware sin detectar	Se bloquea software legítimo de producción	Equilibrio / costo
Retención de clientes	Se pierde un cliente	Se ofrece un descuento innecesario	Recall (si el descuento es barato)
Alertas del SOC	Intrusión sin atender	Analistas saturados de alertas	Precisión con recall mínimo

$$C = c_{FN} \cdot FN + c_{FP} \cdot FP$$

BLOQUE 4 · 2:15–2:45

Fuga de datos

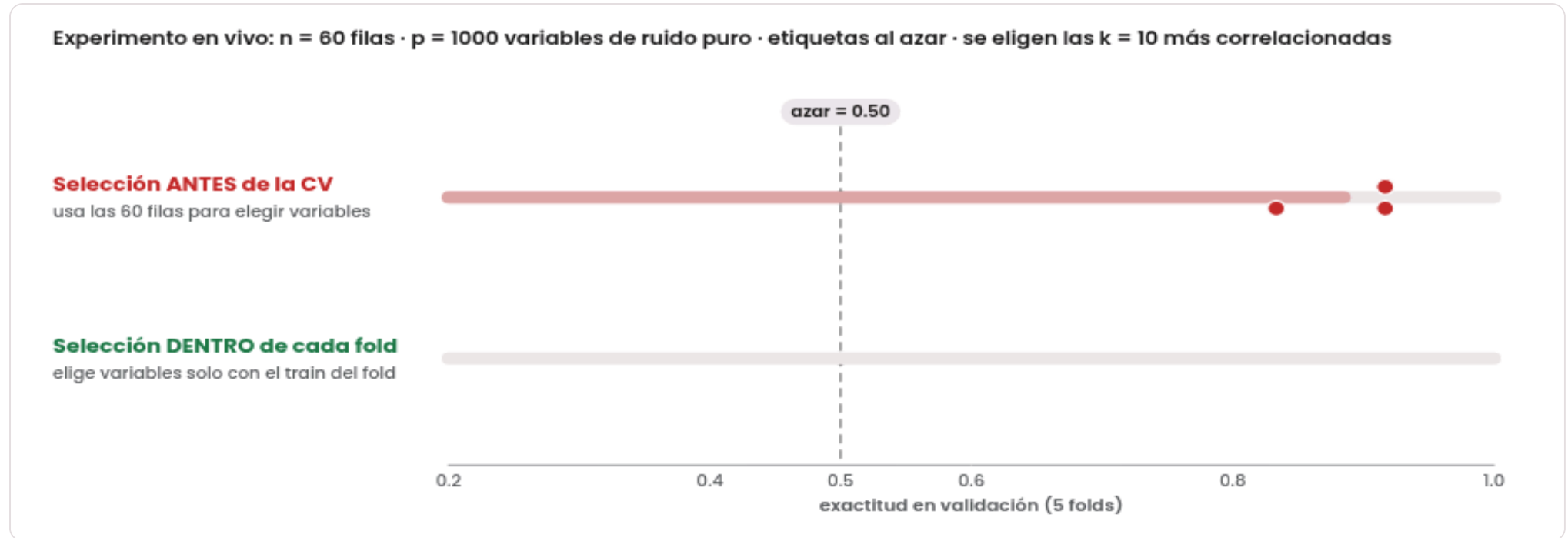
Cuando el modelo “ve” el examen antes de rendirlo.

Cuatro formas de hacer trampa sin querer

- 1 **Preprocesar antes de partir:** escalar, imputar o seleccionar variables con todo el dataset.
- 2 **Variables del futuro:** datos que no existen al momento de predecir (`boat` en el Titanic).
- 3 **Duplicados entre particiones:** el mismo cliente o documento en train y en test.
- 4 **Decidir mirando el test:** ajustar hiperparámetros o umbrales con el conjunto de prueba.

Síntoma típico: resultados **demasiado buenos** que se desploman en producción.

Exactitud de 0.88 sobre ruido puro



Todo lo que aprende de los datos, dentro del Pipeline

Python

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_validate

num = Pipeline([("imp", SimpleImputer(strategy="median")),
                 ("esc", StandardScaler())])
prep = ColumnTransformer([("num", num, num_cols),
                          ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)])
modelo = Pipeline([("prep", prep),
                   ("clf", LogisticRegression(max_iter=1000))])
res = cross_validate(modelo, X_train, y_train, cv=5,
                     scoring=["f1", "average_precision"])
```

Flujo end-to-end reproducible para un caso de TI

- Datos: **Telco Customer Churn** (OpenML 42178).
- Calidad de datos y prueba separada primero.
- `Pipeline` + `ColumnTransformer`, baseline y dos modelos.
- Validación cruzada estratificada (media $\pm$ desv.).
- Métrica y umbral según una matriz de costos.
- Errores por segmento y limitaciones.
- IA agéntica o de chat permitida, **declarada**.

Rúbrica (100 pts)

Reproducibilidad 15 · Calidad y sin fuga 20 ·
Pipeline 15 · Baseline y validación 15 ·
Métricas y costo 15 · Errores y limitaciones 10
· Comunicación 10

Antes de irte, responde en el chat

- 1 ¿Qué métrica usarías para un detector de intrusiones de red en el que el 0.5 % del tráfico es malicioso? ¿Por qué no la exactitud?
- 2 Describe una posible fuga de datos en un problema de tu trabajo.

A CONTINUACIÓN · LABORATORIO GUIADO DE 1.5 H

Ahora, manos al código

E0 · entorno reproducible

E1.3 · fuga de datos y pipelines

Tarea 1 · vie 25/09